

HTML Forms & Accessibility

Input with purpose.

Create forms that are clear to complete, practical to validate, and available to people using any device or input method.

We will build each part in HTML first: labels, types, grouping, native validation, error feedback, and keyboard access.

A USEFUL INTERACTION

YOUR NAME

EMAIL ADDRESS

PROGRAMME CHOICE

SEND APPLICATION

A form is a conversation contract

HTML defines what a person can enter, how the browser packages it, and where that information should go. Every useful control has a purpose, a name, and a clear destination.

1

A person provides a value

A label explains the question. An input collects one meaningful answer.

|

2

The browser names the value

The name attribute becomes the key sent with the learner's answer.

|

3

A destination receives it

`action` says where to send data; `method` says how to send it.

If the information has no clear question, name, or destination, the form is incomplete.

A MINIMAL INTEREST FORM

```
<form action="/interest" method="post">
  <label for="email">Email address</label>
  <input id="email" name="email"
    type="email" required>

  <button type="submit">Request programme
  details</button>
</form>
```

action

The URL that receives the completed data.

method

post sends data for a change or submission; **get** usually expresses a search or filter in the URL.

Labels make controls understandable

Visual proximity is not enough. Screen readers and assistive technologies require a strict programmatic connection to know what a form field is asking for.

The Explicit Connection

The `for` attribute on a `<label>` must exactly match the `id` attribute on the `<input>`. This binds them together in the accessibility tree.

Expanded Click Targets

When a label is properly bound, clicking the text automatically focuses the input. This is a massive usability win, especially for checkboxes.

Placeholders Are Not Labels

Placeholders disappear the moment a user starts typing, destroying context. They also frequently fail minimum color contrast requirements.

ACCESSIBLE Programmatically Bound

```
<label for="firstName">First name</label>
<input type="text" id="firstName" name="firstName">
```

INACCESSIBLE Visual Proximity Only

```
<div class="label-text">First name</div>
<input type="text" name="firstName">
```

INACCESSIBLE Placeholder as Label

```
<input type="text" name="firstName"
        placeholder="First name">
```

Choose a precise input type

The type attribute does more than change how an input looks. It fundamentally changes how mobile devices display keyboards, how browsers validate data, and how password managers autofill forms.

Contextual Keyboards

On mobile devices, specific types trigger optimized keyboards. `type="email"` shows the "@" and ".com" keys, while `type="tel"` shows a large numeric dialpad.

Native Validation

Browsers automatically check if the data matches the type. A `type="url"` input will reject plain text that isn't a valid web address, requiring zero JavaScript.

Better Autofill & Security

Using `type="password"` masks characters and signals password managers to save or generate credentials securely.

COMMON INPUT TYPES

```
<!-- Shows @ and .com on mobile -->
<input type="email" name="email">

<!-- Shows numeric keypad -->
<input type="tel" name="phone">

<!-- Masks characters, triggers password managers -->
<input type="password" name="pwd">

<!-- Native calendar picker -->
<input type="date" name="dob">

<!-- Requires a valid web address -->
<input type="url" name="website">
```

Group related choices

Group related options so assistive tech can announce the question before individual answers.

`<fieldset>`

A structural container that binds related inputs so browsers and screen readers understand their shared context.

`<legend>`

The caption or question describing the group; it should be the first child of the fieldset.

GROUPING RADIO BUTTONS

```
<fieldset>
  <legend>Preferred learning format</legend>
  <input type="radio" id="fmt-online" name="format">
<label for="fmt-online">Online (Remote)</label>
  <input type="radio" id="fmt-campus" name="format">
<label for="fmt-campus">On Campus (Nairobi)</label>
</fieldset>
```

BROWSER DEFAULT RENDER

Preferred learning format

- ☒ Online (Remote)
- ☐ On Campus (Nairobi)

Let HTML prevent obvious mistakes

Browsers have built-in rules to catch errors before data ever reaches the server. Use native attributes to guide users without writing custom JavaScript.

required

Prevents form submission if the field is empty.

minlength / maxlength

Enforces character count limits for text inputs.

min / max

Sets numeric or date boundaries (e.g., age must be at least 16).

pattern

Validates the value against a regular expression (e.g., specific ID formats).

inputmode

Triggers the correct mobile keyboard (e.g., numeric) without changing validation rules.

ENROLLMENT FORM VALIDATION

```
<form action="/enroll" method="post">
  <!-- Required & Length -->
  <label for="username">Username</label>
  <input id="username" name="username" type="text" required
    minlength="3" maxlength="20">
  <!-- Pattern & Inputmode -->
  <label for="studentid">Student ID (e.g., LEA-1234)
  </label>
  <input id="studentid" name="studentid" type="text"
    pattern="LEA-\d{4}" inputmode="text" required>
  <!-- Min & Max -->
  <label for="age">Age</label>
  <input id="age" name="age" type="number" min="16"
    max="99" required>
  <button type="submit">Enroll</button>
</form>
```

Errors must explain the next step

When validation fails, color alone is not enough. We must programmatically connect the error message to the input and clearly explain how to fix it.

`aria-invalid="true"`

Forces assistive technologies to announce that the field's current state is invalid, regardless of visual styling.

`aria-describedby="[id]"`

Programmatically connects the error message text to the input field so they are read together when the user focuses the input.

DON'T RELY ON COLOR

"Invalid email" is unhelpful. "Include an '@' symbol in the email address" gives the user an actionable next step.

ACCESSIBLE ERROR STATE

```
<label for="email">Email address</label>
<input type="email" id="email" name="email"
      value="student#example.com"
      aria-invalid="true"
      aria-describedby="email-error">

<div id="email-error" class="error-text">
  Error: Email must include an '@' symbol.
</div>
```

BROWSER RENDER

Email address

student#example.com

! Error: Email must include an '@' symbol.

Buttons must say what happens

A button is the final commitment in a form. Its type dictates its behavior, and its text must clearly describe the action that will occur when clicked.

`type="submit"`

The default behavior inside a form. It gathers all the input data and sends it to the server URL defined in the form's `action` attribute.

`type="reset"`

Instantly clears all form fields. **Avoid using this.** It is almost exclusively clicked by accident, destroying the user's hard work.

`type="button"`

Does absolutely nothing natively. It is used to trigger custom JavaScript actions (like opening a modal or adding a new row) without submitting the form.

ACCESSIBLE BUTTON COPY

VAGUE (AVOID)

Submit

Click Here

Go

DESCRIPTIVE (USE)

Create Account

Send Message

Pay KES 45,000

CLEAR INTENT IN HTML

```
<!-- Submits the form -->
```

```
<button type="submit"> Enroll in Software  
Engineering </button>
```

```
<!-- Triggers a JS function, no submission -->
```

```
<button type="button" id="add-course"> Add another  
course </button>
```


Keyboard access is a product requirement

Many users navigate the web using only a keyboard. If a control cannot be reached with the Tab key or activated with the Enter key, it does not exist for them.

Logical Source Order

The Tab key follows the HTML document structure, not the visual CSS layout. Keep the DOM order logical so focus moves predictably from top to bottom.

Visible Focus Indicators

Never remove the browser's default focus outline (outline: none) without providing a highly visible, custom alternative. Users must know where they are.

Semantic Controls

Native `<button>` and `<a>` elements are keyboard-accessible by default. Generic elements like `<div>` and `<span>` are ignored by the Tab key.

THE SKIP LINK PATTERN

```
<!-- Allows keyboard users to bypass long navs -->  
<a href="#main-content" class="skip-link">  
  Skip to main content  
</a>  
  
<nav>...</nav>  
<main id="main-content">...</main>
```

NATIVE VS. FAKE CONTROLS

```
<!-- GOOD: Focusable, responds to Enter/Space -->  
<button type="button">Save Changes</button>  
  
<!-- BAD: Invisible to keyboards, click-only -->  
<div class="btn" onclick="save()">Save Changes</div>
```

Assemble one accessible registration form

Bringing it all together: explicit labels, precise input types, semantic grouping, native validation, and clear button intent.

EXPLICIT LABELS

Every input has a matching `<label for="...">`.

PRECISE TYPES

Use `type="email"` for proper keyboards.

SEMANTIC GROUPING

Radio buttons wrapped in `<fieldset>` and `<legend>`.

NATIVE VALIDATION

`required` and `minlength` prevent bad data.

CLEAR ACTION

Descriptive submit button text, not just "Go".

COMPLETE REGISTRATION FORM

```
<form action="/register" method="post">
  <fieldset><legend>Student Information</legend>
  <label for="fname">Full Name</label>
  <input type="text" id="fname" name="fname" required minlength="2">
  <label for="email">Email Address</label>
  <input type="email" id="email" name="email" required>
</fieldset>
  <fieldset><legend>Campus Preference</legend>
  <input type="radio" id="camp-nbo" name="campus" value="nairobi"
required>
  <label for="camp-nbo">Nairobi Campus</label>
  <input type="radio" id="camp-rmt" name="campus" value="remote">
  <label for="camp-rmt">Remote Learning</label>
</fieldset>
  <button type="submit">Complete Registration</button>
</form>
```

Test before you call it accessible

Accessibility is not a theoretical checklist; it is an experience you must verify. Before shipping any form, run it through this four-step manual testing loop to ensure it works for everyone.

01 The Keyboard Test

Unplug your mouse. Can you reach every single field using only the Tab key? Can you select radio buttons with Space and submit the form with Enter? Is the focus ring always clearly visible?

GOAL: 100% KEYBOARD OPERABLE

02 The Label Check

Click the text of every label on the screen. Does it immediately focus the corresponding input field? If it doesn't, your for and id attributes are broken, missing, or mismatched.

GOAL: PROGRAMMATIC BINDING

03 The Validation Trap

Try to submit the form completely empty. Enter letters into number fields. Enter an invalid email address. Does the browser catch the errors and clearly explain how to fix them?

GOAL: CLEAR ERROR RECOVERY

04 The Zoom & Contrast Review

Zoom your browser to 200%. Does the layout break or hide inputs? Check your colors: are error messages and placeholder texts readable against their backgrounds?

GOAL: VISUAL RESILIENCE

Forms lab: help people finish

Build an accessible, robust registration form for LEA Labs that guides users to success without frustration or confusion.

DELIVERABLES

- Build a complete course registration form using semantic HTML.
- Use explicit `<label>` bindings for every single input field.
- Apply precise type attributes (email, tel, date) for better keyboards.
- Group related options (like course selection) using `<fieldset>` and `<legend>`.

SUCCESS CHECKS

- Can you navigate through every field and submit the form using **only the Tab and Enter keys**?
- Does clicking on the text of a label automatically focus its corresponding input?
- Does the browser block submission if required fields are empty or invalid?

EXTENSION CHALLENGE

- Implement an accessible error feedback state for at least one field.
- Use `aria-invalid="true"` to signal the error programmatically.
- Use `aria-describedby` to connect the input to a descriptive error message explaining how to fix the issue.